LOT Pager ??? 04-FIRMWARE-SPEC

04 · Firmware Specification & Companion Software

1. Stack

- ESP32-S3, **ESP-IDF 5.x** (C/C++), FreeRTOS. (PlatformIO/Arduino OK for P2 prototype.)
- TLS via mbedTLS with pinned lot-systems.com CA; OTA via `esp_https_ota`, dual-slot, signed (secure boot v2 at PVT).
- Flash map: bootloader · NVS (token, Wi-Fi) · 2× OTA app · SPIFFS (fonts, splash) · coredump.

2. Modules

Module	Responsibility
`net`	Wi-Fi provisioning (BLE/QR), reconnect, time sync
`lotapi`	REST + WebSocket client for the LOT API (spec in 05)
`inbox`	Receives notifications; stores last 20; ack/read state
`ui`	Renders text cards: title, body (≤ 80 chars), icon, battery/Wi-Fi glyphs; auto-dim via VEML7700
`button`	Debounce, short press = **Copy** (current card → `device_copy` event), long press (2 s) = menu/pairing
`sensors`	BME688/BSEC, BMI270, VEML7700 every 15–30 min, batch upload
`camera`	Off by default; on-demand snapshot or local inference; LED forced on while active
`power`	BQ25185/MAX17048, light-sleep between polls, deep-sleep on low battery
`ota` / `diag`	Update, logs ring buffer, serial console

3. State machine

`BOOT → PROVISION (no token) → CONNECT → IDLE (light sleep, WS keepalive 30 s) → NOTIFY (wake, buzz, show, 20 s timeout) → IDLE`; Copy press in NOTIFY → POST event → toast “Copied ✓”.

4. Pager behavior

- Priority levels: `info` (silent), `nudge` (haptic once), `urgent` (haptic ×3 + LED).
- Quiet hours enforced server-side AND on-device.

- Offline: queue Copy events (max 50) with timestamps; flush on reconnect.

5. Power budget (typ., 280 mAh)

Light sleep + WS keepalive ≈ 1.2 mA avg → 9.7 days; screen 20 s ×30/day adds ≈ 0.15 mA avg; sensors adds ≈ 0.05 mA. Target ≥ 5 days (leaves margin for Wi-Fi retries). Verify on hardware.

6. Security

Per-device token (256-bit) burned at provisioning, revocable from site; TLS 1.2+; flash encryption + secure boot at PVT; no camera frames leave device without explicit user-enabled setting; camera LED hardwired to camera power rail.

7. LOT Device Tool (software to connect with firmware)

Small cross-platform app (web page using **WebSerial/WebUSB** at lot-systems.com/device, fallback Node CLI): 1. Flash firmware (`esptool-js`) and read version. 2. Provision: Wi-Fi SSID/pass + pairing code → obtains device token. 3. Run factory test (screen pattern, button, sensors, Qi voltage, camera frame) and print PASS/FAIL + serial → used for the 100-unit run. 4. Pull diagnostics log, trigger OTA. Serial protocol: newline-delimited JSON over USB-CDC, e.g. `{"cmd":"test","id":"all"}` → `{"ok":true,"results":{...}}`.

8. Documents this feeds (separate files, later sessions)

Firmware release notes, API change log, factory-test procedure, user manual (PDF), service manual (PDF).